Aim: Considered there are N philosophers seated around a circular table with one chopstick between each pair of philosophers. There is one chopstick between each philosopher. A philosopher may eat if he can pick up the two chopsticks adjacent to him. One chopstick may be picked up by any one of its adjacent followers but not both.Write a program to solve the problem using process synchronization technique.

INPUT:

```
krupa_bawane@LAPTOP-AI4\
  GNU nano 7.2                                   dining.c
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 5

sem_t chopstick[N];

void* philosopher(void* num)
{
    int i = *(int*)num;

    while(1)
    {
        printf("Philosopher %d is Thinking\n", i);
        sleep(1);

        // Pick left chopstick
        sem_wait(&chopstick[i]);
        // Pick right chopstick
        sem_wait(&chopstick[(i+1)%N]);

        printf("Philosopher %d is Eating\n", i);
        sleep(1);

        // Put down chopsticks
        sem_post(&chopstick[i]);
        sem_post(&chopstick[(i+1)%N]);
    }
}

int main()
{
    pthread_t thread[N];
    int phil[N];
                                          [ Read 51 lines ]
^G Help      ^O Write Out   ^W Where Is    ^K Cut       ^T Execute   ^C Location   M-U Undo     M-A Set Mark   M-] To Bracket
^X Exit      ^R Read File   ^\ Replace     ^U Paste     ^J Justify   ^/ Go To Line M-E Redo     M-6 Copy       ^Q Where Was
```

```
krupa_bawane@LAPTOP-AI4\
  GNU nano 7.2                                   dining.c
        printf("Philosopher %d is Thinking\n", i);
        sleep(1);

        // Pick left chopstick
        sem_wait(&chopstick[i]);
        // Pick right chopstick
        sem_wait(&chopstick[(i+1)%N]);

        printf("Philosopher %d is Eating\n", i);
        sleep(1);

        // Put down chopsticks
        sem_post(&chopstick[i]);
        sem_post(&chopstick[(i+1)%N]);
    }
}

int main()
{
    pthread_t thread[N];
    int phil[N];

    for(int i=0;i<N;i++)
        sem_init(&chopstick[i],0,1);

    for(int i=0;i<N;i++)
    {
        phil[i]=i;
        pthread_create(&thread[i],NULL,philosopher,&phil[i]);
    }

    for(int i=0;i<N;i++)
        pthread_join(thread[i],NULL);

    return 0;
}

^G Help      ^O Write Out   ^W Where Is    ^K Cut       ^T Execute   ^C Location   M-U Undo     M-A Set Mark   M-] To Bracket
^X Exit      ^R Read File   ^\ Replace     ^U Paste     ^J Justify   ^/ Go To Line M-E Redo     M-6 Copy       ^Q Where Was
```

OUTPUT:

```
krupa_bawane@LAPTOP-AI4V
nakrupa_bawane@LAPTOP-AI4VE3LU:~$ n
n: command not found
krupa_bawane@LAPTOP-AI4VE3LU:~$ nano dining.c
krupa_bawane@LAPTOP-AI4VE3LU:~$ gcc dining.c -lpthread
krupa_bawane@LAPTOP-AI4VE3LU:~$ ./a.out
Philosopher 0 is Thinking
Philosopher 2 is Thinking
Philosopher 1 is Thinking
Philosopher 3 is Thinking
Philosopher 4 is Thinking
Philosopher 2 is Eating
Philosopher 2 is Thinking
Philosopher 1 is Eating
Philosopher 1 is Thinking
Philosopher 0 is Eating
Philosopher 0 is Thinking
Philosopher 4 is Eating
Philosopher 4 is Thinking
Philosopher 3 is Eating
Philosopher 3 is Thinking
Philosopher 2 is Eating
Philosopher 2 is Thinking
Philosopher 1 is Eating
Philosopher 0 is Eating
Philosopher 1 is Thinking
Philosopher 0 is Thinking
Philosopher 4 is Eating
Philosopher 4 is Thinking
Philosopher 3 is Eating
Philosopher 3 is Thinking
Philosopher 2 is Eating
Philosopher 2 is Thinking
Philosopher 1 is Eating
Philosopher 1 is Thinking
Philosopher 0 is Eating
Philosopher 0 is Thinking
Philosopher 4 is Eating
Philosopher 4 is Thinking
Philosopher 3 is Eating
Philosopher 3 is Thinking
Philosopher 2 is Eating
```